Projects for course in Applied Generative AI

- **Notice the example of the full project overview first.**

Project Overview: Automated Document Summarization and Question Answering

The goal is to build an application that can:

1. **Accept user requests:** Users will provide a document (e.g., a PDF, text file, or URL) and a series of questions related to the document's content.
2. **Process the document:** The application will extract and process the document's text.
3. **Utilize a language model:** The application will use a chosen language model to summarize the document and answer the user's questions based on the document's content.
4. **Present the results:** The application will present the summary and answers to the user in a clear and organized format.

Exe 1: Document Processing and Summarization (Estimated 30 Hours)

Task: Develop a robust system that can handle document uploads (or URL fetches), extract text, and generate a concise summary of the document's content.

Specific Requirements:

- **Document Input:**
    - Implement functionality to accept documents from various sources:
        - Local file upload (PDF, TXT).
        - URL input for web pages.
    - Handle potential errors (e.g., invalid file types, inaccessible URLs).
- **Text Extraction:**
    - Employ libraries or APIs to extract text from PDF documents.
    - Use web scraping techniques (e.g., Beautiful Soup, Scrapy) to extract text from web pages. Or, use an LLM for that.
    - Clean and preprocess the extracted text (e.g., removing unnecessary characters, handling encoding issues).
- **Summarization:**
    - Implement a function that sends the extracted text to a language model for summarization.
    - Allow the user to specify the desired summary length or style (e.g., short, detailed, bullet points).
    - **LLM Options:**

        - Option 1: Gemini.
        - Option 2: OpenAI
- **Output:**
    - Display the generated summary to the user in a clear and readable format.

Exe 2: Question Answering (Estimated 30 Hours)

Task: Extend the application to answer user-provided questions based on the processed document's content.

Specific Requirements:

- **Question Input:**
    - Implement a user interface for entering questions related to the document.
    - Allow multiple questions to be submitted at once.
- **Question Answering:**
    - Develop a function that sends the extracted document text and the user's questions to the language model.
    - Prompt the language model to answer the questions based solely on the provided document content.
    - **LLM Options:**
        - Continue to use the LLM selected in phase 1.
- **Output:**
    - Display the answers to the user's questions in a clear and organized manner, referencing the relevant sections of the document if possible.
    - Display a confidence score of the LLM answers if the LLM provides it.
    - Log all the steps, and errors, to the log file.
- **User Interface:**
    - Create a simple and effective user interface for the whole application. This can be a web interface using Flask or Streamlit or a command line interface.

Team Collaboration:

- Divide the tasks among the four team members.
- Establish clear communication channels and regular meetings.

Grading:

- Functionality and correctness of the application.
- Code quality and organization.
- User interface design.

Important Notes:

- Pay close attention to API rate limits and error handling.
- Ensure that the application handles documents up to 2 MB.
- Focus on creating a user-friendly and intuitive experience.
- Document your code well.
- Test your code well.